
ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation

Adam Paszke

Faculty of Mathematics, Informatics and Mechanics
University of Warsaw, Poland
a.paszke@students.mimuw.edu.pl

Abhishek Chaurasia, Sangpil Kim, Eugenio Culurciello

Electrical and Computer Engineering
Purdue University, USA
aabhish, sangpilkim, euge@purdue.edu

Abstract

The ability to perform pixel-wise semantic segmentation in real-time is of paramount importance in mobile applications. Recent deep neural networks aimed at this task have the disadvantage of requiring a large number of floating point operations and have long run-times that hinder their usability. In this paper, we propose a novel deep neural network architecture named ENet (efficient neural network), created specifically for tasks requiring low latency operation. ENet is up to $18\times$ faster, requires $75\times$ less FLOPs, has $79\times$ less parameters, and provides similar or better accuracy to existing models. We have tested it on CamVid, Cityscapes and SUN datasets and report on comparisons with existing state-of-the-art methods, and the trade-offs between accuracy and processing time of a network. We present performance measurements of the proposed architecture on embedded systems and suggest possible software improvements that could make ENet even faster.

1 Introduction

Recent interest in augmented reality wearables, home-automation devices, and self-driving vehicles has created a strong need for semantic-segmentation (or visual scene-understanding) algorithms that can operate in real-time on low-power mobile devices. These algorithms label each and every pixel in the image with one of the object classes. In recent years, the availability of larger datasets and computationally-powerful machines have helped deep convolutional neural networks (CNNs) [1, 2, 3, 4] surpass the performance of many conventional computer vision algorithms [5, 6, 7]. Even though CNNs are increasingly successful at classification and categorization tasks, they provide coarse spatial results when applied to pixel-wise labeling of images. Therefore, they are often cascaded with other algorithms to refine the results, such as color based segmentation [8] or conditional random fields [9], to name a few.

In order to both spatially classify and finely segment images, several neural network architectures have been proposed, such as SegNet [10, 11] or fully convolutional networks [12]. All these works are based on a VGG16 [13] architecture, which is a very large model designed for multi-class classification. These references propose networks with huge numbers of parameters, and long inference times. In these conditions, they become unusable for many mobile or battery-powered applications, which require processing images at rates higher than 10 fps.

In this paper, we propose a new neural network architecture optimized for fast inference and high accuracy. Examples of images segmented using ENet are shown in Figure 1. In our work, we chose

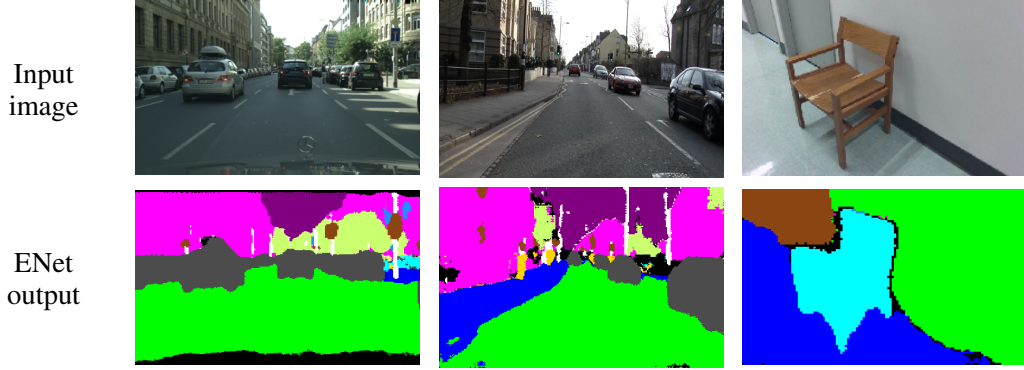


Figure 1: ENet predictions on different datasets (left to right Cityscapes, CamVid, and SUN).

not to use any post-processing steps, which can of course be combined with our method, but would worsen the performance of an end-to-end CNN approach.

In Section 3 we propose a fast and compact encoder-decoder architecture named ENet. It has been designed according to rules and ideas that have appeared in the literature recently, all of which we discuss in Section 4. Proposed network has been evaluated on Cityscapes [14] and CamVid [15] for driving scenario, whereas SUN dataset [16] has been used for testing our network in an indoor situation. We benchmark it on NVIDIA Jetson TX1 Embedded Systems Module as well as on an NVIDIA Titan X GPU. The results can be found in Section 5.

2 Related work

Semantic segmentation is important in understanding the content of images and finding target objects. This technique is of utmost importance in applications such as driving aids and augmented reality. Moreover, real-time operation is a must for them, and therefore, designing CNNs *carefully* is vital. Contemporary computer vision applications extensively use deep neural networks, which are now one of the most widely used techniques for many different tasks, including semantic segmentation. This work presents a new neural network architecture, and therefore we aim to compare to other literature that performs the large majority of inference in the same way.

State-of-the-art scene-parsing CNNs use two separate neural network architectures combined together: an **encoder** and a **decoder**. Inspired by probabilistic auto-encoders [17, 18], encoder-decoder network architecture has been introduced in SegNet-basic [10], and further improved in SegNet [11]. The encoder is a vanilla CNN (such as VGG16 [13]) which is trained to classify the input, while the decoder is used to upsample the output of the encoder [12, 19, 20, 21, 22]. However, these networks are slow during inference due to their large architectures and numerous parameters. Unlike in fully convolutional networks (FCN) [12], fully connected layers of VGG16 were discarded in the latest incarnation of SegNet, in order to reduce the number of floating point operations and memory footprint, making it the smallest of these networks. Still, none of them can operate in real-time.

Other existing architectures use simpler classifiers and then cascade them with Conditional Random Field (CRF) as a post-processing step [9, 23]. As shown in [11], these techniques use onerous post-processing steps and often fail to label the classes that occupy fewer number of pixels in a frame. CNNs can be also combined with recurrent neural networks [20] to improve accuracy, but then they suffer from speed degradation. Also, one has to keep in mind that RNN, used as a post-processing step, can be used in conjunction with any other technique, including the one presented in this work.

3 Network architecture

The architecture of our network is presented in Table 1. It is divided into several stages, as highlighted by horizontal lines in the table and the first digit after each block name. Output sizes are reported for an **example input** image resolution of 512×512 . We adopt a view of ResNets [24] that describes them as having a single main branch and extensions with convolutional filters that separate from it,

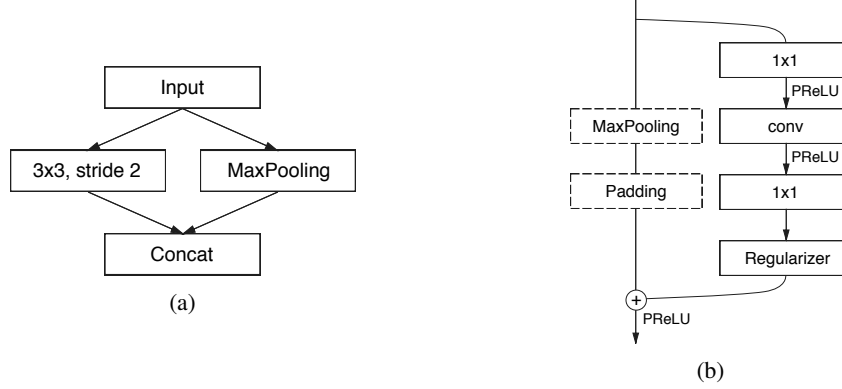


Figure 2: (a) ENet initial block. MaxPooling is performed with non-overlapping 2×2 windows, and the convolution has 13 filters, which sums up to 16 feature maps after concatenation. This is heavily inspired by [28]. (b) ENet bottleneck module. conv is either a regular, dilated, or full convolution (also known as deconvolution) with 3×3 filters, or a 5×5 convolution decomposed into two asymmetric ones.

and then merge back with an element-wise addition, as shown in Figure 2b. Each block consists of three convolutional layers: a 1×1 projection that reduces the dimensionality, a main convolutional layer (conv in Figure 2b), and a 1×1 expansion. We place Batch Normalization [25] and PReLU [26] between all convolutions. Just as in the original paper, we refer to these as bottleneck modules. If the bottleneck is downsampling, a max pooling layer is added to the main branch. Also, the first 1×1 projection is replaced with a 2×2 convolution with stride 2 in both dimensions. We zero pad the activations, to match the number of feature maps. conv is either a regular, dilated or full convolution (also known as deconvolution or fractionally strided convolution) with 3×3 filters. Sometimes we replace it with asymmetric convolution i.e. a sequence of 5×1 and 1×5 convolutions. For the regularizer, we use Spatial Dropout [27], with $p = 0.01$ before bottleneck2.0, and $p = 0.1$ afterwards.

The initial stage contains a single block, that is presented in Figure 2a. Stage 1 consists of 5 bottleneck blocks, while stage 2 and 3 have the same structure, with the exception that stage 3 does not downsample the input at the beginning (we omit the 0th bottleneck). These three first stages are the encoder. Stage 4 and 5 belong to the decoder.

We did not use bias terms in any of the projections, in order to reduce the number of kernel calls and overall memory operations, as cuDNN [29] uses separate kernels for convolution and bias addition. This choice didn't have any impact on the accuracy. Between each convolutional layer and following non-linearity we use Batch Normalization [25]. In the decoder max pooling is replaced with max unpooling, and padding is replaced with spatial convolution without bias. We did not use pooling indices in the last upsampling module, because the initial block operated on the 3 channels of the input frame, while the final output has C feature maps (the number of object classes). Also, for performance reasons, we decided to place only a bare full convolution as the last module of the network, which alone takes up a sizeable portion of the decoder processing time.

Table 1: ENet architecture. Output sizes are given for an example input of 512×512 .

Name	Type	Output size
initial		$16 \times 256 \times 256$
bottleneck1.0	downsampling	$64 \times 128 \times 128$
4× bottleneck1.x		$64 \times 128 \times 128$
bottleneck2.0	downsampling	$128 \times 64 \times 64$
bottleneck2.1		$128 \times 64 \times 64$
bottleneck2.2	dilated 2	$128 \times 64 \times 64$
bottleneck2.3	asymmetric 5	$128 \times 64 \times 64$
bottleneck2.4	dilated 4	$128 \times 64 \times 64$
bottleneck2.5		$128 \times 64 \times 64$
bottleneck2.6	dilated 8	$128 \times 64 \times 64$
bottleneck2.7	asymmetric 5	$128 \times 64 \times 64$
bottleneck2.8	dilated 16	$128 \times 64 \times 64$
<i>Repeat section 2, without bottleneck2.0</i>		
bottleneck4.0	upsampling	$64 \times 128 \times 128$
bottleneck4.1		$64 \times 128 \times 128$
bottleneck4.2		$64 \times 128 \times 128$
bottleneck5.0	upsampling	$16 \times 256 \times 256$
bottleneck5.1		$16 \times 256 \times 256$
fullconv		$C \times 512 \times 512$

4 Design choices

In this section we will discuss our most important experimental results and intuitions, that have shaped the final architecture of ENet.

Feature map resolution Downsampling images during semantic segmentation has two main drawbacks. Firstly, reducing feature map resolution implies loss of spatial information like exact edge shape. Secondly, full pixel segmentation requires that the output has the same resolution as the input. This implies that strong downsampling will require equally strong upsampling, which increases model size and computational cost. The first issue has been addressed in FCN [12] by adding the feature maps produced by encoder, and in SegNet [10] by saving indices of elements chosen in max pooling layers, and using them to produce sparse upsampled maps in the decoder. We followed the SegNet approach, because it allows to reduce memory requirements. Still, we have found that strong downsampling hurts the accuracy, and tried to limit it as much as possible.

However, downsampling has one big advantage. Filters operating on downsampled images have a bigger receptive field, that allows them to gather more context. This is especially important when trying to differentiate between classes like, for example, rider and pedestrian in a road scene. It is not enough that the network learns how people look, the context in which they appear is equally important. In the end, we have found that it is better to use dilated convolutions for this purpose [30].

Early downsampling One crucial intuition to achieving good performance and real-time operation is realizing that processing large input frames is very expensive. This might sound very obvious, however many popular architectures do not pay much attention to optimization of early stages of the network, which are often the most expensive by far.

ENet first two blocks heavily reduce the input size, and use only a small set of feature maps. The idea behind it, is that visual information is highly spatially redundant, and thus can be compressed into a more efficient representation. Also, our intuition is that the initial network layers should not directly contribute to classification. Instead, they should rather act as good feature extractors and only preprocess the input for later portions of the network. This insight worked well in our experiments; increasing the number of feature maps from 16 to 32 did not improve accuracy on Cityscapes [14] dataset.

Decoder size In this work we would like to provide a different view on encoder-decoder architectures than the one presented in [11]. SegNet is a very symmetric architecture, as the encoder is an exact mirror of the decoder. Instead, our architecture consists of a large encoder, and a small decoder. This is motivated by the idea that the encoder should be able to work in a similar fashion to original classification architectures, i.e. to operate on smaller resolution data and provide for information processing and filtering. Instead, the role of the decoder, is to upsample the output of the encoder, only fine-tuning the details.

Nonlinear operations A recent paper [31] reports that it is beneficial to use ReLU and Batch Normalization layers before convolutions. We tried applying these ideas to ENet, but this had a detrimental effect on accuracy. Instead, we have found that removing most ReLUs in the initial layers of the network improved the results. It was quite a surprising finding so we decided to investigate its cause.

We replaced all ReLUs in the network with PReLU [26], which use an additional parameter per feature map, with the goal of learning the negative slope of non-linearities. We expected that in layers where identity is a preferable transfer function, PReLU weights will have values close to 1, and conversely, values around 0 if ReLU is preferable. Results of this experiment can be seen in Figure 3.

Initial layers weights exhibit a large variance and are slightly biased towards positive values, while in the later portions of the encoder they settle to a recurring pattern. All layers in the main branch behave nearly exactly like regular ReLUs, while the weights inside bottleneck modules are negative i.e. the function inverts and scales down negative values. We hypothesize that identity did not work well in our architecture because of its limited depth. The reason why such lossy functions are learned might be that the original ResNets [31] are networks that can be hundreds of layers deep, while our network uses only a couple of layers, and it needs to quickly filter out information. It is notable

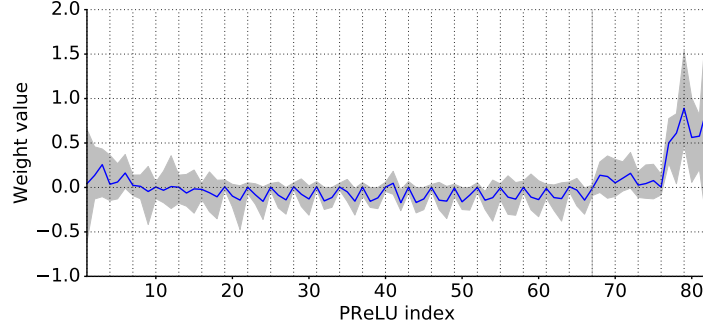


Figure 3: PReLU weight distribution vs network depth. Blue line is the weights mean, while an area between maximum and minimum weight is grayed out. Each vertical dotted line corresponds to a PReLU in the main branch and marks the boundary between each of bottleneck blocks. The gray vertical line at 67th module is placed at encoder-decoder border.

that the decoder weights become much more positive and learn functions closer to identity. This confirms our intuitions that the decoder is used only to fine-tune the upsampled output.

Information-preserving dimensionality changes As stated earlier, it is necessary to downsample the input early, but aggressive dimensionality reduction can also hinder the information flow. A very good approach to this problem has been presented in [28]. It has been argued that a method used by the VGG architectures, i.e. as performing a pooling followed by a convolution expanding the dimensionality, however relatively cheap, introduces a representational bottleneck (or forces one to use a greater number of filters, which lowers computational efficiency). On the other hand, pooling after a convolution, that increases feature map depth, is computationally expensive. Therefore, as proposed in [28], we chose to perform pooling operation in parallel with a convolution of stride 2, and concatenate resulting feature maps. This technique allowed us to speed up inference time of the initial block 10 times.

Additionally, we have found one problem in the original ResNet architecture. When downsampling, the first 1×1 projection of the convolutional branch is performed with a stride of 2 in both dimensions, which effectively discards 75% of the input. Increasing the filter size to 2×2 allows to take the full input into consideration, and thus improves the information flow and accuracy. Of course, it makes these layers $4 \times$ more computationally expensive, however there are so few of these in ENet, that the overhead is unnoticeable.

Factorizing filters It has been shown that convolutional weights have a fair amount of redundancy, and each $n \times n$ convolution can be decomposed into two smaller ones following each other: one with a $n \times 1$ filter and the other with a $1 \times n$ filter [32]. This idea has been also presented in [28], and from now on we adopt their naming convention and will refer to these as asymmetric convolutions. We have used asymmetric convolutions with $n = 5$ in our network, so cost of these two operations is similar to a single 3×3 convolution. This allowed to increase the variety of functions learned by blocks and increase the receptive field.

What’s more, a sequence of operations used in the bottleneck module (projection, convolution, projection) can be seen as decomposing one large convolutional layer into a series of smaller and simpler operations, that are its low-rank approximation. Such factorization allows for large speedups, and greatly reduces the number of parameters, making them less redundant [32]. Additionally, it allows to make the functions they compute richer, thanks to the non-linear operations that are inserted between layers.

Dilated convolutions As argued above, it is very important for the network to have a wide receptive field, so it can perform classification by taking a wider context into account. We wanted to avoid overly downsampling the feature maps, and decided to use dilated convolutions [30] to improve our model. They replaced the main convolutional layers inside several bottleneck modules in the stages that operate on the smallest resolutions. These gave a significant accuracy boost, by raising IoU on Cityscapes by around 4 percentage points, with no additional cost. We obtained the best accuracy

when we interleaved them with other bottleneck modules (both regular and asymmetric), instead of arranging them in sequence, as has been done in [30].

Regularization Most pixel-wise segmentation datasets are relatively small (on order of 10^3 images), so such expressive models as neural networks quickly begin to overfit them. In initial experiments, we used L2 weight decay with little success. Then, inspired by [33], we have tried stochastic depth, which increased accuracy. However it became apparent that dropping whole branches (i.e. setting their output to 0) is in fact a special case of applying Spatial Dropout [27], where either all of the channels, or none of them are ignored, instead of selecting a random subset. We placed Spatial Dropout at the end of convolutional branches, right before the addition, and it turned out to work much better than stochastic depth.

5 Results

We benchmarked the performance of ENet on three different datasets to demonstrate real-time and accurate for practical applications. We tested on CamVid and Cityscapes datasets of road scenes, and SUN RGB-D dataset of indoor scenes. We set SegNet [11] as a baseline since it is one of the fastest segmentation models, that also has way fewer parameters and requires less memory to operate than FCN. All our models, training, testing and performance evaluation scripts were using the Torch7 machine-learning library, with cuDNN backend. To compare results, we use class average accuracy and intersection-over-union (IoU) metrics.

5.1 Performance Analysis

We report results on inference speed on widely used NVIDIA Titan X GPU as well as on NVIDIA TX1 embedded system module. ENet was designed to achieve more than 10 fps on the NVIDIA TX1 board with an input image size 640×360 , which is adequate for practical road scene parsing applications. For inference we merge batch normalization and dropout layers into the convolutional filters, to speed up all networks.

Table 2: Performance comparison.

Model	NVIDIA TX1						NVIDIA Titan X					
	480×320		640×360		1280×720		640×360		1280×720		1920×1080	
	ms	fps	ms	fps	ms	fps	ms	fps	ms	fps	ms	fps
SegNet	757	1.3	1251	0.8	-	-	69	14.6	289	3.5	637	1.6
ENet	47	21.1	69	14.6	262	3.8	7	135.4	21	46.8	46	21.6

Inference time Table 2 compares inference time for a single input frame of varying resolution. We also report the number of frames per second that can be processed. Dashes indicate that we could not obtain a measurement, due to lack of memory. ENet is significantly faster than SegNet, providing high frame rates for real-time applications and allowing for practical use of very deep neural network models with encoder-decoder architecture.

Table 3: Hardware requirements. FLOPs are estimated for an input of $3 \times 640 \times 360$.

	GFLOPs	Parameters	Model size (fp16)
SegNet	286.03	29.46M	56.2 MB
ENet	3.83	0.37M	0.7 MB

Hardware requirements Table 3 reports a comparison of number of floating point operations and parameters used by different models. ENet efficiency is evident, as its requirements are on two orders of magnitude smaller. Please note that we report storage required to save model parameters in half precision floating point format. ENet has so few parameters, that the required space is only 0.7MB, which makes it possible to fit the whole network in an extremely fast on-chip memory in embedded

processors. Also, this alleviates the need for model compression [34], making it possible to use general purpose neural network libraries. However, if one needs to operate under incredibly strict memory constraints, these techniques can still be applied to ENet as well.

Software limitations One of the most important techniques that has allowed us to reach these levels of performance is convolutional layer factorization. However, we have found one surprising drawback. Although applying this method allowed us to greatly reduce the number of floating point operations and parameters, it also increased the number of individual kernels calls, making each of them smaller.

We have found that some of these operations can become so cheap, that the cost of GPU kernel launch starts to outweigh the cost of the actual computation. Also, because kernels do not have access to values that have been kept in registers by previous ones, they have to load all data from global memory at launch, and save it when their work is finished. This means that using a higher number of kernels, increases the number of memory transactions, because feature maps have to be constantly saved and reloaded. This becomes especially apparent in case of non-linear operations. In ENet, PReLUs consume more than a quarter of inference time. Since they are only simple point-wise operations and are very easy to parallelize, we hypothesize it is caused by the aforementioned data movement.

These are serious limitations, however they could be resolved by performing kernel fusion in existing software i.e. create kernels that apply non-linearities to results of convolutions directly, or perform a number of smaller convolutions in one call. This improvement in GPU libraries, such as cuDNN, could increase the speed and efficiency of our network even further.

5.2 Benchmarks

We have used the Adam optimization algorithm [35] to train the network. It allowed ENet to converge very quickly and on every dataset we have used training took only 3-6 hours, using four Titan X GPUs. It was performed in two stages: first we trained only the encoder to categorize downsampled regions of the input image, then we appended the decoder and trained the network to perform upsampling and pixel-wise classification. Learning rate of $5e-4$ and L2 weight decay of $2e-4$, along with batch size of 10 consistently provided the best results. We have used a custom class weighing scheme defined as $w_{\text{class}} = \frac{1}{\ln(c+p_{\text{class}})}$. In contrast to the inverse class probability weighing, the weights are bounded as the probability approaches 0. c is an additional hyper-parameter, which we set to 1.02 (i.e. we restrict the class weights to be in the interval of $[1, 50]$).

Table 4: Cityscapes test set results

Model	Class IoU	Class iIoU	Category IoU	Category iIoU
SegNet	56.1	34.2	79.8	66.4
ENet	58.3	34.4	80.4	64.0

Cityscapes This dataset consists of 5000 fine-annotated images, out of which 2975 are available for training, 500 for validation, and the remaining 1525 have been selected as test set [14]. Cityscapes was the most important benchmark for us, because of its outstanding quality and highly varying road scenarios, often featuring many pedestrians and cyclists. We trained on 19 classes that have been selected in the official evaluation scripts [14]. It makes use of an additional metric called instance-level intersection over union metric (iIoU), which is IoU weighed by the average object size. As reported in Table 4, ENet outperforms SegNet in class IoU and iIoU, as well as in category IoU. ENet is currently the fastest model in the Cityscapes benchmark. Example predictions for images from validation set are presented in Figure 4.

CamVid Another automotive dataset, on which we have tested ENet, was CamVid. It contains 367 training and 233 testing images [15]. There are eleven different classes such as building, tree, sky, car, road, etc. while the twelfth class contains unlabeled data, which we ignore while training. The original frame resolution for this dataset is 960×720 but we downsampled the images to 480×360 before training. In Table 5 we compare the performance of ENet with existing state-of-the-art algorithms.

Table 5: Results on CamVid test set of (1) SegNet-Basic, (2) SegNet, and (3) ENet

Model	Building	Tree	Sky	Car	Sign	Road	Pedestrian	Fence	Pole	Sidewalk	Bicyclist	Class avg.	Class IoU
1	75.0	84.6	91.2	82.7	36.9	93.3	55.0	47.5	44.8	74.1	16.0	62.9	n/a
2	88.8	87.3	92.4	82.1	20.5	97.2	57.1	49.3	27.5	84.4	30.7	65.2	55.6
3	74.7	77.8	95.1	82.4	51.0	95.1	67.2	51.7	35.4	86.7	34.1	68.3	51.3

ENet outperforms other models in six classes, which are difficult to learn because they correspond to smaller objects. ENet output for example images from the test set can be found in Figure 5.

Table 6: SUN RGB-D test set results

Model	Global avg.	Class avg.	Mean IoU
SegNet	70.3	35.6	26.3
ENet	59.5	32.6	19.7

SUN RGB-D The SUN dataset consists of 5285 training images and 5050 testing images with 37 indoor object classes. We did not make any use of depth information in this work and trained the network only on RGB data. In Table 6 we compare the performance of ENet with SegNet [11], which is the only neural network model that reports accuracy on this dataset. Our results, though inferior in global average accuracy and IoU, are comparable in class average accuracy. Since global average accuracy and IoU are metrics that favor correct classification of classes occupying large image patches, researchers generally emphasize the importance of other metrics in case of semantic segmentation. One notable example is introduction of iIoU metric [14]. Comparable result in class average accuracy indicates, that our network is capable of differentiating smaller objects nearly as well as SegNet. Moreover, the difference in accuracy should not overshadow the huge performance gap between these two networks. ENet can process the images in real-time, and is nearly $20\times$ faster than SegNet on embedded platforms. Example predictions from SUN test set are shown in Figure 6.

6 Conclusion

We have proposed a novel neural network architecture designed from the ground up specifically for semantic segmentation. Our main aim is to make efficient use of scarce resources available on embedded platforms, compared to fully fledged deep learning workstations. Our work provides large gains in this task, while matching and at times exceeding existing baseline models, that have an order of magnitude larger computational and memory requirements. The application of ENet on the NVIDIA TX1 hardware exemplifies real-time portable embedded solutions.

Even though the main goal was to run the network on mobile devices, we have found that it is also very efficient on high end GPUs like NVIDIA Titan X. This may prove useful in data-center applications, where there is a need of processing large numbers of high resolution images. ENet allows to perform large-scale computations in a much faster and more efficient manner, which might lead to significant savings.

Acknowledgment

This work is partly supported by the Office of Naval Research (ONR) grants N00014-12-1-0167, N00014-15-1-2791 and MURI N00014-10-1-0278. We gratefully acknowledge the support of NVIDIA Corporation with the donation of the TX1, Titan X, K40 GPUs used for this research.

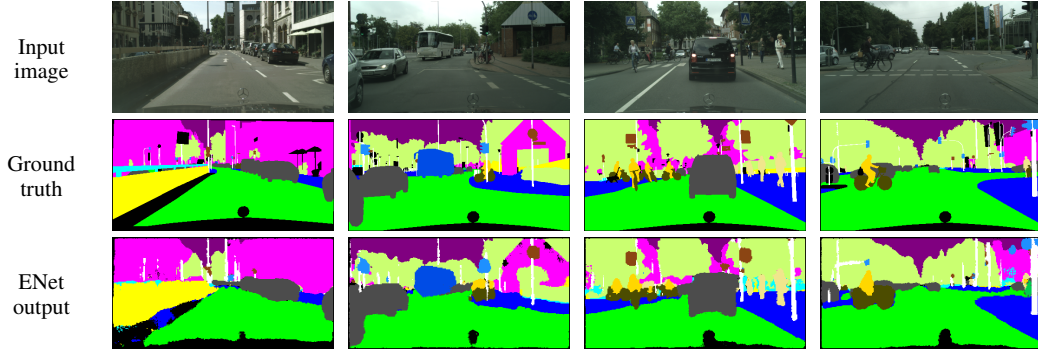


Figure 4: ENet predictions on Cityscapes validation set [14]

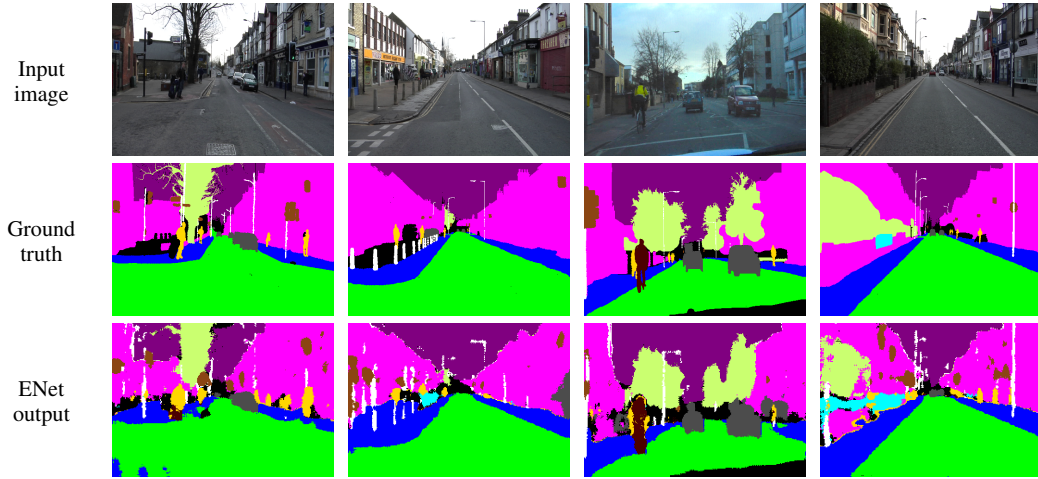


Figure 5: ENet predictions on CamVid test set [15]

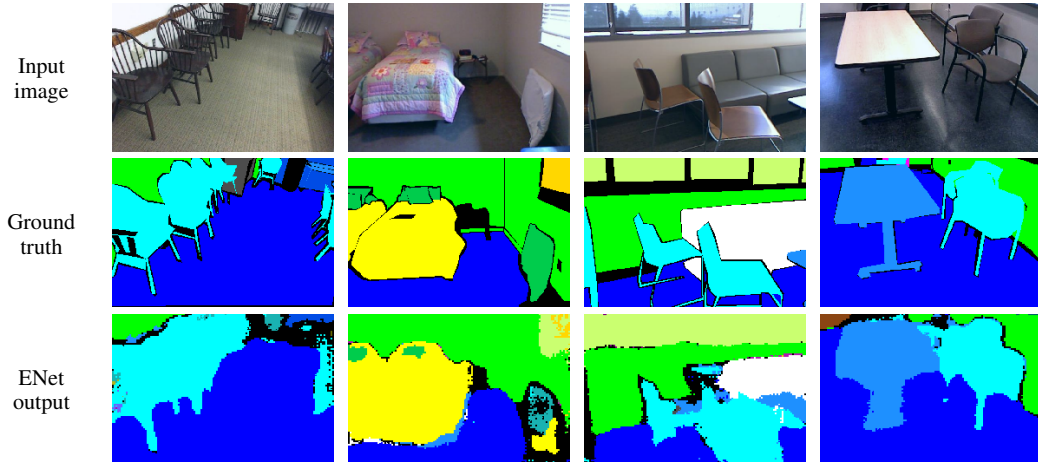


Figure 6: ENet predictions on SUN RGB-D test set [16]

References

- [1] Y. LeCun and Y. Bengio, “Convolutional networks for images, speech, and time series,” *The handbook of brain theory and neural networks*, pp. 255–258, 1998.
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems 25*, 2012, pp. 1097–1105.
- [3] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.

- [4] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 1–9.
- [5] J. Shotton, J. Winn, C. Rother, and A. Criminisi, "Textonboost for image understanding: Multi-class object recognition and segmentation by jointly modeling texture, layout, and context," *Int. Journal of Computer Vision (IJCV)*, January 2009.
- [6] F. Perronnin, Y. Liu, J. Sánchez, and H. Poirier, "Large-scale image retrieval with compressed fisher vectors," in *Computer Vision and Pattern Recognition (CVPR), 2010 IEEE Conference on*, 2010, pp. 3384–3391.
- [7] K. E. A. van de Sande, J. R. R. Uijlings, T. Gevers, and A. W. M. Smeulders, "Segmentation as selective search for object recognition," in *IEEE International Conference on Computer Vision*, 2011.
- [8] C. Farabet, C. Couprie, L. Najman, and Y. LeCun, "Learning hierarchical features for scene labeling," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1915–1929, Aug 2013.
- [9] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "Semantic image segmentation with deep convolutional nets and fully connected crfs," *arXiv preprint arXiv:1412.7062*, 2014.
- [10] V. Badrinarayanan, A. Handa, and R. Cipolla, "Segnet: A deep convolutional encoder-decoder architecture for robust semantic pixel-wise labelling," *arXiv preprint arXiv:1505.07293*, 2015.
- [11] V. Badrinarayanan, A. Kendall, and R. Cipolla, "Segnet: A deep convolutional encoder-decoder architecture for image segmentation," *arXiv preprint arXiv:1511.00561*, 2015.
- [12] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3431–3440.
- [13] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [14] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The cityscapes dataset for semantic urban scene understanding," in *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [15] G. J. Brostow, J. Shotton, J. Fauqueur, and R. Cipolla, "Segmentation and recognition using structure from motion point clouds," in *ECCV (1)*, 2008, pp. 44–57.
- [16] S. Song, S. P. Lichtenberg, and J. Xiao, "Sun rgb-d: A rgb-d scene understanding benchmark suite," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 567–576.
- [17] M. A. Ranzato, F. J. Huang, Y.-L. Boureau, and Y. LeCun, "Unsupervised learning of invariant feature hierarchies with applications to object recognition," in *Computer Vision and Pattern Recognition, 2007. CVPR'07. IEEE Conference on*, 2007, pp. 1–8.
- [18] J. Ngiam, A. Khosla, M. Kim, J. Nam, H. Lee, and A. Y. Ng, "Multimodal deep learning," in *Proceedings of the 28th international conference on machine learning (ICML-11)*, 2011, pp. 689–696.
- [19] H. Noh, S. Hong, and B. Han, "Learning deconvolution network for semantic segmentation," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 1520–1528.
- [20] S. Zheng, S. Jayasumana, B. Romera-Paredes, V. Vineet, Z. Su, D. Du, C. Huang, and P. H. Torr, "Conditional random fields as recurrent neural networks," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 1529–1537.
- [21] D. Eigen and R. Fergus, "Predicting depth, surface normals and semantic labels with a common multi-scale convolutional architecture," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 2650–2658.
- [22] S. Hong, H. Noh, and B. Han, "Decoupled deep neural network for semi-supervised semantic segmentation," in *Advances in Neural Information Processing Systems*, 2015, pp. 1495–1503.
- [23] P. Sturgess, K. Alahari, L. Ladicky, and P. H. Torr, "Combining appearance and structure from motion features for road scene understanding," in *BMVC 2012-23rd British Machine Vision Conference*, 2009.
- [24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *arXiv preprint arXiv:1512.03385*, 2015.
- [25] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *arXiv preprint arXiv:1502.03167*, 2015.
- [26] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," pp. 1026–1034, 2015.
- [27] J. Tompson, R. Goroshin, A. Jain, Y. LeCun, and C. Bregler, "Efficient object localization using convolutional networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 648–656.
- [28] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," *arXiv preprint arXiv:1512.00567*, 2015.
- [29] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cudnn: Efficient primitives for deep learning," *arXiv preprint arXiv:1410.0759*, 2014.
- [30] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," *arXiv preprint arXiv:1511.07122*, 2015.
- [31] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," *arXiv preprint arXiv:1603.05027*, 2016.
- [32] J. Jin, A. Dundar, and E. Culurciello, "Flattened convolutional neural networks for feedforward acceleration," *arXiv preprint arXiv:1412.5474*, 2014.
- [33] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Weinberger, "Deep networks with stochastic depth," *arXiv preprint arXiv:1603.09382*, 2016.
- [34] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural network with pruning, trained quantization and Huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.
- [35] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.